

STAP++ 二维平面弹性单元扩展报告

Q4 四边形单元与 T3 三角形单元

有限元法大作业 I：编程训练

2026 年 5 月 29 日

摘要

本文面向有限元法大作业 I 的“扩展 STAP++ 程序功能”要求，在原 STAP++ 框架中实现了两类二维平面弹性单元：Q4 四节点等参四边形单元和 T3 三节点常应变三角形单元。程序支持平面应力和平面应变两种材料模式，保持 STAP++ 原有输入输出风格，并完成材料读取、单元刚度装配、总体方程求解和单元应力输出。验证部分包括基础算例、分片试验、网格加密自收敛分析、原 Bar 单元回归测试以及非法输入测试。结果表明，Q4 与 T3 均能通过常应变分片试验，拉伸与剪切算例随网格加密呈现合理收敛趋势，新增功能未破坏原有 Bar 单元。

目录

1 引言	3
2 程序结构与输入格式	3
2.1 代码扩展位置	3
2.2 材料输入格式	3
2.3 单元输入格式	4
3 算法说明	4
3.1 平面弹性本构矩阵	4
3.2 Q4 四节点等参单元	4
3.3 T3 三节点常应变单元	5
4 实现方案	5
5 使用方法	5
6 程序验证与确认	6
6.1 基础算例与回归测试	6

目录	2
6.2 分片试验	6
6.3 网格加密自收敛分析	6
6.4 异常输入测试	7
7 任务分工与完成情况	8
8 版本控制与仓库说明	8
9 结论	8

1 引言

课程项目要求在 STAP++ 或 STAP90 基础上扩展有限元程序功能。个人作业至少需要增加一个单元，并给出单元基本原理、编程思路、输入数据格式、分片试验、收敛性分析和验证算例结果。为使作业内容更完整，本文没有只停留在单个 Q4 单元，而是在 Q4 的基础上进一步实现 T3 单元，使程序具备两类常用二维平面弹性单元。

本项目的主要功能如下：

- 实现 Q4 四节点等参四边形单元，采用 2×2 Gauss 积分；
- 实现 T3 三节点常应变三角形单元，采用解析常应变矩阵；
- 支持平面应力和平面应变两种本构矩阵；
- 输出每个二维单元的 σ_x 、 σ_y 和 τ_{xy} ；
- 提供 Q4 与 T3 的分片试验、网格加密验证和异常输入测试；
- 使用 Git 分支和 tag 存档原 Q4 版本，便于回滚。

2 程序结构与输入格式

2.1 代码扩展位置

本项目尽量保持 STAP++ 原结构不变，只按已有 Bar 单元风格增加新类和分支。核心修改包括：

- 新增 CQ4 和 CT3 单元类，分别位于 `src/h` 与 `src/cpp`；
- 在 `Material.h/.cpp` 中新增 `CQ4Material` 和 `CT3Material`；
- 在 `ElementGroup` 中根据单元类型分配对应的元素数组和材料数组；
- 在 `Outputter` 中增加 Q4/T3 的材料、单元连接关系和应力输出；
- 修正元素材料指针的所有权问题，避免析构时释放非拥有指针；
- 修正材料读取失败未向上传递的问题，使非法材料模式能正常报错。

2.2 材料输入格式

Q4 与 T3 采用相同的材料输入格式：

```
nset E nu thickness mode
```

其中 E 为弹性模量， ν 为泊松比，`thickness` 为单元厚度，`mode=1` 表示平面应力，`mode=2` 表示平面应变。若 `mode` 不是 1 或 2，或厚度不为正，程序会返回错误。

2.3 单元输入格式

二维单元均只使用节点的 x 、 y 坐标和 u_x 、 u_y 两个自由度。由于 STAP++ 节点类原本包含三个平移自由度，所有二维算例都将 u_z 约束，避免出现无刚度自由度。

Q4 单元组使用 NPAR(1)=2:

```
2 NUME NUMMAT
nset E nu thickness mode
element_id node1 node2 node3 node4 material_set
```

T3 单元组使用 NPAR(1)=3:

```
3 NUME NUMMAT
nset E nu thickness mode
element_id node1 node2 node3 material_set
```

两类单元均要求节点按逆时针方向输入。Q4 检查 Gauss 点的 $\det J$ ，T3 检查面积 A ；若几何方向错误或单元退化，程序报错停止。

3 算法说明

3.1 平面弹性本构矩阵

平面应力本构矩阵为

$$\mathbf{D} = \frac{E}{1 - \nu^2} \begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & (1 - \nu)/2 \end{bmatrix}. \quad (1)$$

平面应变本构矩阵为

$$\mathbf{D} = \frac{E}{(1 + \nu)(1 - 2\nu)} \begin{bmatrix} 1 - \nu & \nu & 0 \\ \nu & 1 - \nu & 0 \\ 0 & 0 & (1 - 2\nu)/2 \end{bmatrix}. \quad (2)$$

单元应变向量和应力向量分别为

$$\boldsymbol{\varepsilon} = \begin{bmatrix} \varepsilon_x & \varepsilon_y & \gamma_{xy} \end{bmatrix}^T, \quad \boldsymbol{\sigma} = \mathbf{D}\boldsymbol{\varepsilon}. \quad (3)$$

3.2 Q4 四节点等参单元

Q4 单元采用自然坐标 (ξ, η) 中的双线性形函数:

$$N_1 = \frac{1}{4}(1 - \xi)(1 - \eta), \quad N_2 = \frac{1}{4}(1 + \xi)(1 - \eta), \quad (4)$$

$$N_3 = \frac{1}{4}(1 + \xi)(1 + \eta), \quad N_4 = \frac{1}{4}(1 - \xi)(1 + \eta). \quad (5)$$

等参映射由 $x = \sum N_i x_i$ 、 $y = \sum N_i y_i$ 给出。程序在每个 Gauss 点计算 Jacobi 矩阵和应变-位移矩阵 B ，并用 2×2 Gauss 积分计算刚度矩阵：

$$K_e = \sum_{g=1}^4 B_g^T D B_g \det(J_g) t. \quad (6)$$

应力输出取单元中心点 $(\xi, \eta) = (0, 0)$ 的结果。

3.3 T3 三节点常应变单元

T3 单元面积为

$$A = \frac{1}{2} [(x_2 - x_1)(y_3 - y_1) - (x_3 - x_1)(y_2 - y_1)]. \quad (7)$$

定义

$$b_i = y_j - y_k, \quad c_i = x_k - x_j, \quad (8)$$

其中 (i, j, k) 循环取 $(1, 2, 3)$ 、 $(2, 3, 1)$ 、 $(3, 1, 2)$ 。T3 的应变-位移矩阵为常量：

$$B = \frac{1}{2A} \begin{bmatrix} b_1 & 0 & b_2 & 0 & b_3 & 0 \\ 0 & c_1 & 0 & c_2 & 0 & c_3 \\ c_1 & b_1 & c_2 & b_2 & c_3 & b_3 \end{bmatrix}. \quad (9)$$

因此单元刚度矩阵为

$$K_e = t A B^T D B. \quad (10)$$

T3 的应变和应力在单元内为常量，这使其实现简单、稳定，适合作为第二个扩展单元。

4 实现方案

程序实现遵循“最小改动”原则。单元对象负责读取节点编号和材料编号、生成位置矩阵、计算单元刚度和单元应力；材料对象负责读取和输出材料参数；单元组对象负责按类型分配派生类数组；输出对象负责输出单元定义和应力结果。

Q4 和 T3 均复用 STAP++ 的 skyline 总刚度矩阵和 LDLT 求解器，不改变全局装配和求解流程。这样做的优点是风险小，原 Bar 单元行为保持不变；缺点是二维单元必须显式约束 u_z ，否则会出现没有刚度贡献的自由度。

5 使用方法

在 Windows PowerShell 下可按如下方式构建和运行：

```
cmake -S src -B build-codex
cmake --build build-codex --config Debug
.\build-codex\Debug\stap++.exe data\q4-single.dat
.\build-codex\Debug\stap++.exe data\t3_generated\t3-single.dat
```

自动化验证脚本位于 `others/tools/`。生成和解析 T3 验证数据的命令为：

```
python others/tools/t3_generate_cases.py
python others/tools/t3_parse_results.py
```

Q4 对应脚本为 `q4_generate_cases.py` 和 `q4_parse_results.py`。

6 程序验证与确认

6.1 基础算例与回归测试

本项目运行了原 Bar 算例、已有 Q4 算例和新增 T3 算例。Bar 回归用于确认原功能未被破坏；Q4/T3 基础算例用于检查读入、装配、求解、应力输出是否形成闭环。运行结果显示，右侧受拉节点的水平位移为正，拉伸算例的平均 σ_x 与外力方向一致，剪切算例的平均 τ_{xy} 与施加载荷方向一致。

6.2 分片试验

分片试验采用 2×2 网格和线性位移场

$$u = ax, \quad v = by, \quad (11)$$

其中 $a = 1.0 \times 10^{-3}$, $b = -5.0 \times 10^{-4}$ 。为避免修改 STAP++ 的非零位移边界输入格式，验证脚本在程序外部组装刚度矩阵，并用 $\mathbf{f} = \mathbf{K}\mathbf{u}$ 反算等效节点力。该方法可以检验单元能否准确再现常应变状态。

表 1: Q4 与 T3 分片试验结果

单元	最大 u_x 误差	最大 u_y 误差	σ_x 平均误差	σ_y 平均误差
Q4	0.0	0.0	1.54×10^{-4}	4.62×10^{-5}
T3	0.0	0.0	1.54×10^{-4}	4.62×10^{-5}

理论应力为 $\sigma_x = 1.961538 \times 10^2$ 、 $\sigma_y = -4.615385 \times 10^1$ 、 $\tau_{xy} = 0$ 。表中的应力误差主要来自输出文件保留五位科学计数法导致的截断；位移误差为零，说明 Q4 与 T3 均通过常应变分片试验。

6.3 网格加密自收敛分析

网格加密算例包括拉伸和剪切两类。左边界固定，右边界施加等效集中节点力。由于本阶段不再进行 ABAQUS 对比，收敛性以 8×8 网格结果作为参考值进行自收敛比较。

表 2: 拉伸算例网格加密结果

单元	网格	最大 u_x	平均 σ_x	相对差异
Q4	1×1	4.62010×10^{-3}	1.00000×10^3	2.14×10^{-2}
Q4	2×2	4.68107×10^{-3}	1.00000×10^3	8.44×10^{-3}
Q4	4×4	4.71071×10^{-3}	9.99999×10^2	2.16×10^{-3}
Q4	8×8	4.72091×10^{-3}	1.00000×10^3	0
T3	1×1	4.85511×10^{-3}	9.99999×10^2	2.28×10^{-2}
T3	2×2	4.86852×10^{-3}	1.00000×10^3	2.56×10^{-2}
T3	4×4	4.79445×10^{-3}	1.00000×10^3	9.98×10^{-3}
T3	8×8	4.74706×10^{-3}	1.00000×10^3	0

表 3: 剪切算例网格加密结果

单元	网格	最大 u_y	平均 τ_{xy}	相对差异
Q4	1×1	1.10053×10^{-2}	5.00000×10^2	3.51×10^{-1}
Q4	2×2	1.39833×10^{-2}	5.00000×10^2	1.75×10^{-1}
Q4	4×4	1.60605×10^{-2}	5.00000×10^2	5.27×10^{-2}
Q4	8×8	1.69541×10^{-2}	5.00000×10^2	0
T3	1×1	8.58918×10^{-3}	5.00000×10^2	4.73×10^{-1}
T3	2×2	1.14415×10^{-2}	5.00000×10^2	2.99×10^{-1}
T3	4×4	1.45211×10^{-2}	5.00000×10^2	1.10×10^{-1}
T3	8×8	1.63122×10^{-2}	5.00000×10^2	0

结果表明, Q4 和 T3 在拉伸与剪切算例中均随网格加密逐渐接近参考值。T3 在粗网格下与 Q4 有差异, 这是因为 T3 为常应变三角形单元, 同一正方形区域需要两个三角形近似; 加密后其结果同样趋于稳定。

6.4 异常输入测试

异常测试用于确认程序能够发现明显错误输入:

- Q4 非法材料模式: `q4-invalid-mode.dat`;
- Q4 反向节点顺序导致 $\det J \leq 0$: `q4-invalid-jacobian.dat`;
- T3 非法材料模式: `t3-invalid-mode.dat`;
- T3 反向节点顺序导致 $A \leq 0$: `t3-invalid-area.dat`。

上述算例均按预期报错停止, 说明材料模式检查和单元几何检查有效。

7 任务分工与完成情况

本项目为个人大作业,未采用团队分工。本人独立完成以下内容:阅读课程项目要求和 STAP++ 原程序;设计并实现 Q4 与 T3 二维平面弹性单元;编写材料输入、单元读入、刚度矩阵、应力恢复和异常检查代码;生成并运行 Bar 回归、Q4 验证和 T3 验证算例;整理 CSV 数据表;撰写并编译最终报告;使用 Git 进行版本控制和 GitHub 提交。

8 版本控制与仓库说明

项目使用 Git 进行版本控制,仓库来源和提交位置如下:

- 提交仓库: <https://github.com/Albert041118/STAPpp>;
- 原始上游程序: <https://github.com/xzhang66/STAPpp>;
- 本地上游远端: `upstream` 指向课程指定 STAP++ 仓库;
- 主线分支: `main`;
- T3 开发分支: `codex/add-t3-element`。

仓库根目录按课程要求保留 `src`、`make`、`data`、`doc` 和 `others`。其他课程资料、原始参考文档和辅助脚本统一放入 `others`。

为便于从 T3 扩展回滚到原 Q4 最终版本,在新增 T3 前已创建备份 tag:

<code>backup/q4-final-before-t3</code>
--

T3 开发分成两个主要提交:第一个提交实现 T3 单元代码和框架接入,第二个提交加入 T3 验证算例、CSV 表格和报告更新。最终修改已合并并推送到 GitHub 主分支。

9 结论

本文完成了 STAP++ 的二维平面弹性功能扩展。Q4 单元体现了等参映射、Jacobi 检查和数值积分;T3 单元体现了常应变三角形公式、面积检查和与 Q4 的对比验证。两类单元均支持平面应力和平面应变,均通过常应变分片试验,并在拉伸和剪切网格加密算例中表现出合理的自收敛趋势。相比只实现一个 Q4 单元,Q4+T3 的组合更完整地展示了二维有限元单元实现、验证和程序扩展能力。

参考资料

1. 张雄,有限元法基础课程大作业说明,2026。
2. X. Zhang, T. Wang, Y. Liu, *Computational Dynamics*, Tsinghua University Press.
3. STAP++ 源程序: <https://github.com/xzhang66/STAPpp>。